C++

기본

LoveisGreatandHoly

이규훈
010-3446-7374
v_star@naver.com

Programming Language

- C++ 언어의 지향점

절차형
패러다임

객체 지향
패러다임

Programming Language

- 프로그래밍 언어의 Schema

Input ⇨ Processing ⇨ Output

연산(Operator)
조건(Condition)
반복(Loop)

Variable

- type
- void
- string

자료형	크기(byte)	표현 범위
bool	1	true / false
char	1	-128 ~ 127
unsigned char	1	0 ~ 255
short	2	-32,768 ~ 32,767
unsigned short	2	0 ~ 65,535
int	4	-2,147,483,648 ~ 2,147,483,647 기본
unsigned int	4	0 ~ 4,294,967,295 u, U
long	4 또는 8	(4byte 기준) -2,147,483,648 ~ 2,147,483,647 l, L
unsigned long	4 또는 8	0 ~ 4,294,967,295 ul, UL
long long	8	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807 ll, LL
unsigned long long	8	0 ~ 18,446,744,073,709,551,615 ull, ULL
float	4	약 ±3.4 × 10^38 (유효숫자 약 7자리) f, F
double	8	약 ±1.7 × 10^308 (유효숫자 약 15자리) 기본
long double	8 / 12 / 16	시스템에 따라 다름 l, L

Variable

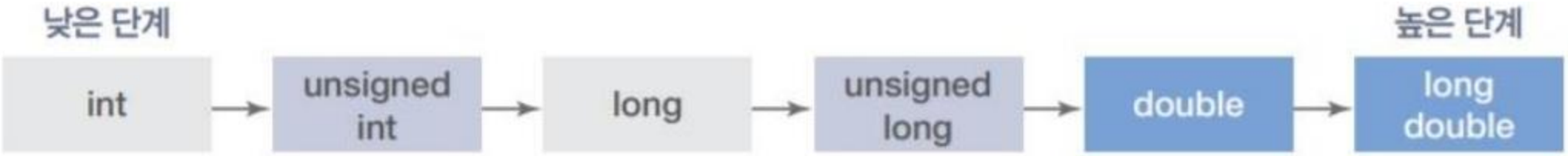
※ CPU가 효율적으로 처리할 수 있는 기본 정수 단위가 보통 32비트(int)이기 때문

- 암묵적 자료형 승격

규칙	원래 자료형	승격된 자료형
1	bool	int
2	char	int
3	short	int
3	unsigned short	unsigned int
4	float	double

예) `bool x = true;  count << x + 100;`

- 암묵적 자료형 변경



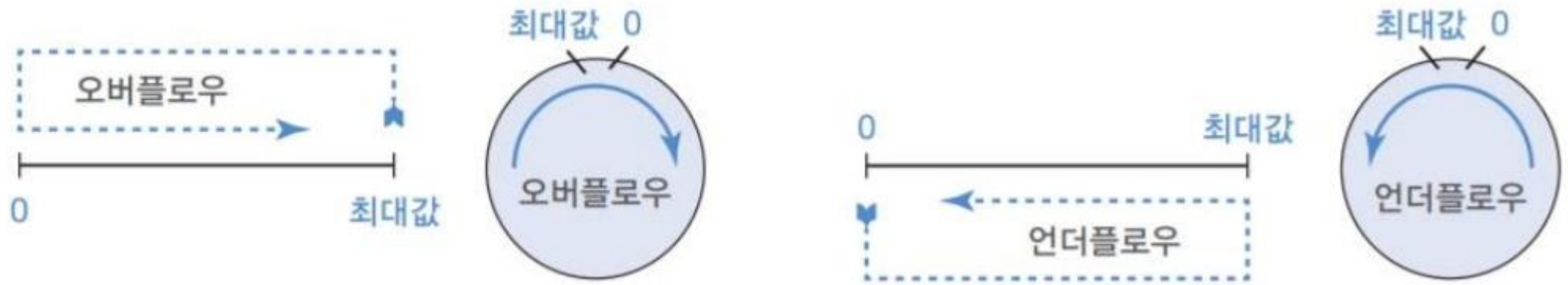
Variable

- 명시적 자료형 변환(캐스팅)
  - 예제

```
double x = 23.56;
int y = 30;
cout << “암묵적 자료형 변환:” << x+y << endl;  //53.56
cout << “명시적 자료형 변환:” << static_cast<int>(x)+y; //53
```

Variable

- 부호있는 오버플로우, 언더플로우



- 부동 소수점의 오버플로우, 언더플로우
  - 오버플로우는 양의 무한대로 수렴, 언더플로우는 음의 무한대 수렴

```
double num1 = +numeric_limits<double>::max();
double num2 = -numeric_limits<double>::max();
cout << num1 * 1000;  // inf
cout << num2 * 1000;  // -inf
```

Variable

- 상수

```
const double PI = 3.14;
```

- 전역변수 : 정수는 0, 부동 소수점은 0.0, 불은 0, 문자도 0
- 지역변수 : 선언하고 초기화 안하면 쓰레기 값 가짐.

Processing

- 연산
- <<
  stream insertion operator

연산자 종류	연산자	설명	예
산술 연산자	+ - * / %	기본 수학 연산	a + b
증감 연산자	++ --	값 1 증가/감소	i++, --i
관계 연산자	== != > < >= <=	값 비교	a > b
논리 연산자	&&, ||, !		!`
대입 연산자	=	값 대입	a = 10
복합 대입 연산자	+= -= *= /= %=	연산 후 대입	a += 5
조건 연산자	? :	삼항 연산자	a>b ? a : b
비트 연산자	& | ^ ~ << >>	비트 단위 연산	a & b
주소 연산자	&	변수 주소 반환	&a
포인터 연산자	*	포인터 참조	*p
크기 연산자	sizeof	메모리 크기 반환	sizeof(int)
형 변환 연산자	(type) static_cast 등	자료형 변환	(int)3.14
멤버 접근 연산자	. ->	구조체/클래스 접근	obj.x
범위 지정 연산자	::	네임스페이스 지정	std::cout
인덱스 연산자	[]	배열 접근	arr[0]
함수 호출 연산자	()	함수 실행	func()
쉼표 연산자	,	여러 식 순차 실행	a=1, b=2

Processing

- 연산

출력조정자	설명	예	출력 결과
endl	줄바꿈 + 버퍼 비우기	cout << "A" << endl;	A (다음 줄)
setw(n)	출력 폭 지정	cout << setw(5) << 10;	" 10"
setfill(c)	빈 공간 채우기	cout << setfill('*') << setw(5) << 10;	***10
left	왼쪽 정렬	cout << left << setw(5) << 10;	10
right	오른쪽 정렬	cout << right << setw(5) << 10;	10
fixed	고정 소수점 표시	cout << fixed << 3.14;	3.140000
setprecision(n)	정밀도 설정	cout << setprecision(3) << 3.14159;	3.14 또는 3.142
showpoint	소수점 항상 표시	cout << showpoint << 5.0;	5.00000
hex	16진수 출력	cout << hex << 255;	ff
dec	10진수 출력	cout << dec << 255;	255
oct	8진수 출력	cout << oct << 255;	377
showbase	진수 접두어 표시	cout << showbase << hex << 255;	0xff
uppercase	대문자 출력	cout << uppercase << hex << 255;	FF
noshowbase	진수 접두어 제거	cout << noshowbase;	기본값

Processing

- 연산
- 출력조정 예제)
  - boolalpha, oct, hex, bin

```
cout << 1237 << endl; //1237
cout << oct << 1237 << endl; //2325
cout << hex << 1237 << endl; //4d5
```

  - showbase, bin

```
cout << bitset<8>(num) << endl; // 00001010
cout << showbase << dec << 1237 << endl;  //1237
cout << showbase << oct << 1237 << endl;  //02325
cout << showbase << hex << 1237 << endl; //0x4d5
```

Processing

- 연산
- 출력조정 예제)

```
double a = 10.0;
double b = 10.5;
std::cout << "기본 출력: " << a << ", " << b << std::endl; // 10, 10.5
// showpoint 적용
std::cout << std::showpoint;
std::cout << "showpoint 적용: " << a << ", " << b << std::endl;
// 10.0000, 10.5000

// 고정 소수점 및 정밀도 조합
std::cout << std::fixed << std::setprecision(2);
std::cout << "fixed + precision(2): " << a << ", " << b << std::endl;
// 10.00, 10.50
```

Processing

- 조건
  - if (조건) 문장;
  - if (조건) { 문장들… }
  - if (조건) { 문장; } else { 문장;}
  - if (조건) {문장;}else if(조건) {문장;} else {문장;}
  - 중첩 if
  - switch case 문
  - 삼항 연산자

Processing

- 반복 => 규칙성, (초기, 조건, 증감 3가지 요인을 반드시 확인!)
- for(초;조;증) {문장;}

```
int sum = 0;
for(int i=0;i<10;i++) {
   sum += i;
}
cout << “sum:” << sum << endl;
return 0;
```

```
int num;
int total = 0;
// 숫자만 입력이 끝날 때반복 (EOF)
while (cin >> num) {
   total += num;
}
cout << "총합: " << total << endl;
return 0;
```

Processing

- 반복 => 규칙성, (초기, 조건, 증감 3가지 요인을 반드시 확인!)

➢while (조건) { 문장; }

➢while (num != -1) {문장;} //sentinel 제어 while 반복문

➢while (cin >> num) {문장;} //EOF 제어 while 반복문

➢while (infile >> num) {문장;} //EOF 제어 while 반복문

➢do while (조건) { 문장; }

➢중첩 반복문

Processing

- 반복

- return : 함수, 반복문에서 사용하면 종료
- break : 반복문에서 멈춤
- continue : 반복문에서 멈추고 조건 확인

function

- 장점 : 재사용, 분할처리, 오류확인
- 함수 구조

```
리턴 자료형 함수 이름(매개변수)
{
   본문
}
```

function

- 종류 : 라이브러리 함수, 사용자 정의 함수

1. 라이브러리 수학, 문자, 시간, 램덤

```
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;

int main() {
    for(int i = 10; i >= 1; i--) {
        cout << i << endl;
        this_thread::sleep_for(chrono::seconds(1));
    }
    cout << "Start!" << endl;
    return 0;
}
```

function

2. 사용자 정의 함수

```
void 함수명 ( )
{
  본문
  return;
}
```

```
void 함수명 ( 매개변수1, ... )
{
  본문
  return;
}
```

```
자료형 함수명 ( )
{
  본문
  return value;
}
```

```
자료형 함수명 ( 매개변수1, ... )
{
  본문
  return value;
}
```

function

2. 사용자 정의 함수와 원형 함수

```
#include <iostream>
using namespace std;

// 선언(프로토타입)
int input();
bool process(int year);
void output(int year, bool result);

int main()
{
  // Input, processing, output
  int year = input();
  bool result = process(year);
  output(year, result);
  return 0;
}
```

function

3. 사용자 정의 함수 : 자료 전달

① 값으로 전달(pass-by-value)

```
#include <iostream>
using namespace std;

void fun(int y); // 함수 선언

int main()
{
  int x = 10; // 변수 선언
  // x를 arguments fun 함수 호출
  fun(x); // x의 값이 변했는지 확인(변하지 않음)
  cout << "main 함수 내부의 x = " << x << endl;
  return 0;
}
```

function

3. 사용자 정의 함수 : 자료 전달

② 참조로 전달(pass-by-reference)

```
#include <iostream>
using namespace std;

// 함수 선언
void fun(int&  y); // & 기호는 y가 참조라는 것을 나타냄

int main()
{
  int x = 10; // 변수 선언
  // x를 argument fun 함수 호출
  fun(x);
  // x의 값이 변했는지 확인(변함)
  cout << "main 함수 내부의 x = " << x << endl;
  return 0;
}
```

function

3. 사용자 정의 함수 : 자료 전달

③ 포인터로 전달(pass-by-pointer) : 인수로 메모리 주소를 매개변수에 전달

C++에서의 Pass-by-Pointer

```
void swapByPointer(int* ptr1, int* ptr2) {
   // 포인터는 NULL 체크가 필요함
   if (ptr1 == nullptr || ptr2 == nullptr)
return;
   int temp = *ptr1;
   *ptr1 = *ptr2;
   *ptr2 = temp;
}

// 호출 시
swapByPointer(&a, &b);
```

C++에서의 Pass-by-Reference (C++ 전용)

```
void swapByReference(int& ref1, int& ref2) {
    // 내부에서 * 기호 없이 일반 변수처럼 사용
    int temp = ref1;
    ref1 = ref2;
    ref2 = temp;
}

// 호출 시 (마치 값을 넘기는 것처럼 보임)
swapByReference(a, b);
```

function

[※ 참고 C 언어에서 포인터]

1. swap(&a, &b) 호출: a의 주소(예: 0x100)와 b의 주소(예: 0x104)가 swap 함수로 복사됩니다.
2. swap 함수 내부: ptr1과 ptr2는 실시간으로 main에 있는 a와 b의 주소를 가리킴 Point.
3. 역참조(*): *ptr1을 수정하면 포인터가 가리키는 실제 주소인 main 함수의 변수 a 값이 변경.

```
#include <stdio.h>
void swap(int *ptr1, int *ptr2) {
    int temp = *ptr1;  // ptr1이 가리키는 주소의 값을 temp에 저장
    *ptr1 = *ptr2;     // ptr2가 가리키는 주소의 값을 ptr1의 주소에 덮어씀
    *ptr2 = temp;      // temp에 저장했던 값을 ptr2의 주소에 저장
}
int main() {
    int a = 32767;    int b = -32768;
    printf("바꾸기 전: a = %d, b = %d\n", a, b);
    swap(&a, &b); // 변수의 '주소(&)'를 함수에 전달합니다.
    printf("바꾸기 후: a = %d, b = %d\n", a, b);
    return 0;
}
```

function

3. 사용자 정의 함수 : 자료 리턴

① 값으로 리턴(return-by-value)

```
#include <iostream>
using namespace std;

bool isEven(int y); // 함수 선언

int main()
{
  cout << boolalpha << isEven(5) << endl;
  cout << boolalpha << isEven(10);
  return 0;
}
bool isEven(int y)
{
  return ((y % 2) == 0);
}
```

function

3. 사용자 정의 함수 : 자료 리턴

② 참조로 전달(return-by-reference) :
※복사로 인한 비용을 줄이려면, 참조로 리턴하는 것이 좋다.

```
int global_val = 100;
int& getGlobal() {   // 반환형 뒤에 &를 붙여 참조를 반환함을 명시합니다.
    return global_val; // 복사본이 아닌 실제 변수의 참조를 반환
}
int main() {
    std::cout << "수정 전: " << global_val << std::endl;
    // 함수 호출 결과에 값을 대입! (Return-by-value라면 불가능)
    getGlobal() = 200;
    std::cout << "수정 후: " << global_val << std::endl;
    return 0;
}
```

③ 포인터로 전달(return-by-pointer) : 거의 사용하지 않음.

function

- 기본 매개변수 지정하기

```
// 함수 선언: 두 번째 매개변수의 기본 값이 40
double calcEarnings(double rate, double hours = 40);

int main()
{
  // 첫 번째 함수 호출이 기본 매개변수를 사용하고 있음
  cout << "직원1의 임금: " << calcEarnings(22.0) << endl;
  cout << "직원2의 임금: " << calcEarnings(12.50, 18);
  return 0;
}
double calcEarnings(double rate, double hours)
{
  double pay;
  pay = hours * rate;
  return pay;
}
```

function

- 함수 Overloading 조건
  - 매개변수(자료형, 개수, 순서)가 달라야 한다.

```
int max ( int a, int b )
{   }
```

```
int max ( double a, double b )
{  }
```

  - 리턴타입은 상관이 없다.

```
int get ( )
{
  본문
  return;
}
```

```
double get ( )
{
  본문
  return;
}
```

  - 이름이 같아야 한다

function

- 변수 Scope
  1. 지역(local)
     - 스코프 겹침
     - 중첩 블록
     - 지역 스코프의 shadowing

```
#include <iostream>
using namespace std;

int main()
{
  int sum = 5;
  cout << sum << endl;
  {
    int sum = 3;
    cout << sum << endl; // 내부 블록의 sum 함수를 사용
  }
  cout << sum << endl;   // 외부 블록의 sum 함수를 사용
  return 0;
}
```

function

- 변수 Scope
  2. 전역(global)

- 전역 스코프의 shadowing

```
#include <iostream>
using namespace std;
int num = 5;   // 전역 변수
int main() {
  cout << num << endl;
  int num = 25;
  cout << num;  // 셰도잉
  return 0;
}
```

- 범위 해결 연산자 ::

```
#include <iostream>
using namespace std;

int num = 5;  // 전역 변수

int main()
{
  int num = 25;  // 지역 변수
  cout << "전역 변수 num의 값 = " << ::num << endl;
  cout << "지역 변수 num의 값 = " << num << endl;
  return 0;
}
```

function

- 정적 지역 변수 (Static Local Variable)

```
void fun();
int main()
{
  fun();
  fun();
  fun();
  return 0;
}
 void fun(){
  int num = 3;   // 암묵적 자동 지역 변수
  int count = 0; // 명시적 자동 지역 변수
  num++;
  count++;
  cout << num << “/” << count << endl;
}
```

```
void fun();
int main()
{
  fun();
  fun();
  fun();
  return 0;
}
void fun()
{
  static int count = 0; // 명시적 정적 변수
  count++;
  cout << "count = " << count << endl;
}
```

function

- 함수 Scope
  1. 함수 이름의 scope
     - 함수는 선언 시점부터 프로그램 마지막까지 scope를 가짐.
     - 선언 함수가 맨 위에 있으면 prototype 함수를 별도 선언 안해도 됨.
  2. 함수 매개변수의 scope
     - 매개변수의 범위는 함수 내부에 한정되는 지역 변수.
     - 함수가 호출될 때 생성되고, 함수가 종료될 때 소멸 => 자동 지역 변수

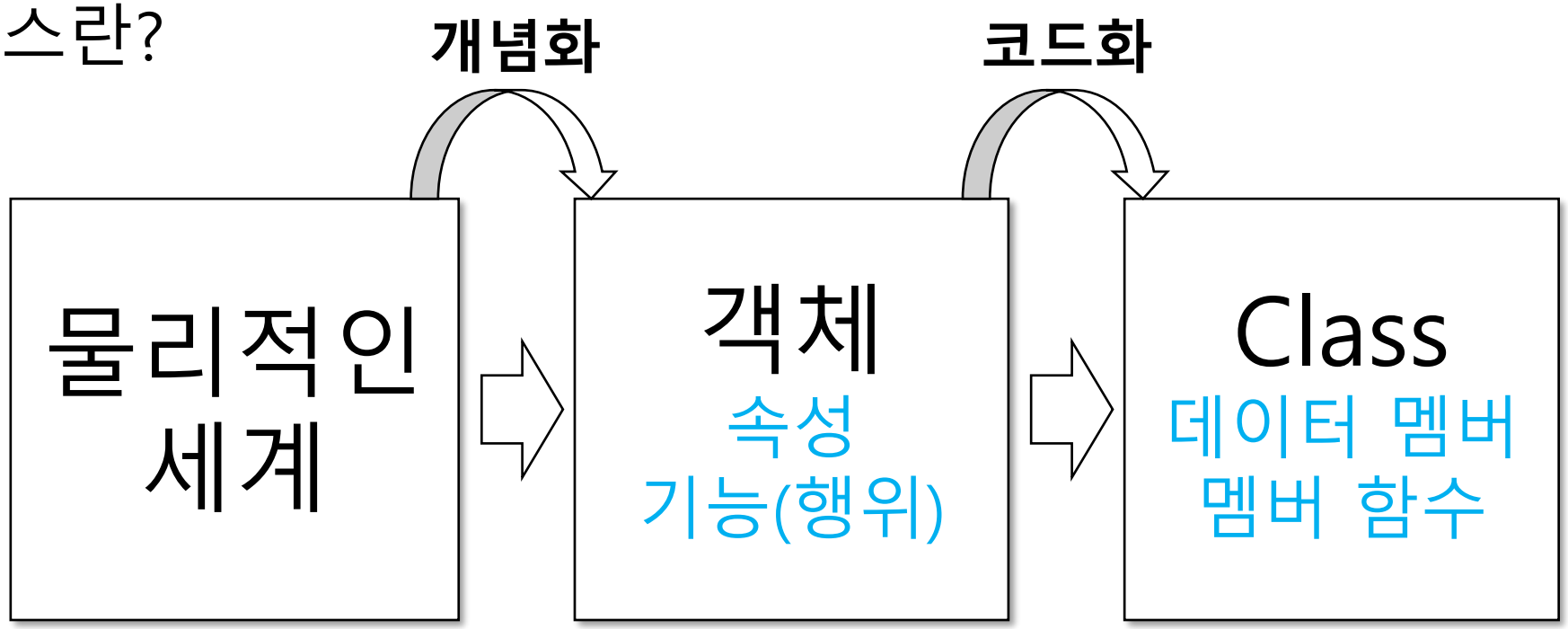
function

- 변수의 초기화
- 자동지역변수는 미초기화시 쓰레기값 가짐.
- 전역변수와 정적지역변수는 기본값으로 초기화 됨.

```
int global;   // 전역 변수

int main()
{
  static int sLocal;   // 정적 지역 변수
  int aLocal;     // 자동 지역 변수 vs에서는 에러발생
  // 출력
  cout << "전역 변수 = " << global << endl;
  cout << "정적 지역 변수 = " << sLocal << endl;
  cout << "자동 지역 변수 = " << aLocal << endl;
  return 0;
}
```

class

- 클래스란?



class

언어 비교

```
#include <stdio.h>
int main()
{
   printf("Hello World \n");
}
```

⇨ C

```
#include <iostream>
int main()
{
   std::cout << "Hello, World!" << std::endl;
}
```

⇨ C++

```
class HelloWorld {
   public void static main(String[] args){
      System.out.println("Hello World");
   }
}
```

⇨ JAVA

class

- 타입을 통해서 인스턴스를 생성할 수 있다.
- 타입은 추상화된 것이고 인스턴스는 타입을 실체화 함.
  - ✓ 데이터 멤버 : 속성, 인스턴스가 가지는 고유의 특징
  - ✓ 멤버 함수 : 기능(행위), 인스턴스가 가지는 고유의 기능
- 클래스 구조

```
class Account     // 헤더
{
  private:
    int money;  // 데이터 멤버 선언
  public:
    void setMoney(int m); // 멤버 함수 선언, 함수 내부에 변경 허가할 때
    int getMoney() const; // 멤버 함수 선언, 함수 내부에 변경 불가할 때
}; //클래스 정의 끝에는 반드시 semi-colon
```

class

- 접근제한자 – 데이터 멤버와 멤버 함수에 기본으로 private 적용
- 데이터 멤버에는 일반적으로 private 사용
- 멤버 함수에는 일반적으로 public 사용(내부에서만 사용 private)

접근 제한자	같은 클레스 內	서브 클래스에서만	모든 곳에서 접근
private	O	X	X
protected	O	O	X
public	O	O	O

class

- 멤버 데이터 (Member Data)
  - ✓ C++11 부터는 클래스 선언 시 직접 데이터 멤버에 값 넣는 것 허용.
  - ✓ 인플레이스 초기화(In-class member initializer)

```
class Player {
public:
    int totalPlayers; // 선언
    Player() { totalPlayers++; }
};

int main() {
    Player p1, p2, p3;
    cout << "플레이어 수: "
    << p1.totalPlayers << endl;
    // -858993459
    return 0;
}
```

```
class Circle {
private:
    double radius = 1.0; // 에러 아님! 기본값으로 설정.
    int count{ 0 };      // 중괄호{} 초기화도 가능
public:
    Circle() {}          // radius는 1.0이 됨
    Circle(double r) : radius(r) {}
    // 생성자에서 r을 넣으면 1.0은 무시되고 r로 초기화됨
};
```

class

- 정적 멤버 변수 (Static Member Variable)
- static 멤버 변수는 클래스 선언 내부에서 초기화 안됨이 원칙.

```
class Player {
public:
    static int totalPlayers; // 선언
    Player() { totalPlayers++; }
};

// 클래스 외부에서 반드시 초기화 (중요!)
int Player::totalPlayers = 0;

int main() {
    Player p1, p2, p3;
    cout << "현재 플레이어 수: " << Player::totalPlayers << endl; // 3
    return 0;
}
```

class

- 멤버 함수 선언 - 방법 1

```
class Member {
private:
    std::string name;     int age;

public:
    void setInfo(std::string n, int a) { name = n;        age = a;  }
    void display() {
        std::cout << "이름: " << name << ", 나이: " << age << std::endl;
    }
};
int main() {
    Member m1; // 객체 생성
    m1.setInfo("홍길동", 25);
    m1.display
    return 0;
}
```

class

- 멤버 함수 선언 - 방법 2 (실무, 대규모 프로젝트에서 간결하게)
- 해당 함수가 어떤 클래스의 소속인지 범위 지정 연산자 :: 사용

```
class Member {
private:
   string name;
   int age;

public:
   // 함수의 프로토타입(원형)만 선언
   void setInfo(std::string n, int a);
   void display();
};
```

```
void Member::setInfo(string n, int a) {
    name = n;     age = a;
}
void Member::display() {
    std::cout << name << “/” << age << endl;
}
int main() {
    Member m1;
    m1.setInfo("이순신", 30);
    m1.display();
    return 0;
}
```

class

- 정적 멤버 함수 (Static Member Method)

```
class Math {
public:
    static int add(int a, int b) { return a + b; }
};

int main() {
    // Math m; 객체 생성 필요 없음!
    cout << "합계: " << Math::add(10, 20) << endl;
    return 0;
}
```

class

- 멤버 함수 선언 – 한정자 const

```
double Circle::getRadius() const {
   return radius;
}

void Circle::setRadius(double radius) {// Setter: 값을 수정해야 하므로 const 제거
   this->radius = radius;  // 멤버변수와 매개변수가 같다면 this->로 구분할 수 있다.
}

void Member::setInfo(string n, int a) {// Setter: 값을 수정해야 하므로 const 제거
    name = n;     age = a;
}

void Member::display() const {  // class 밖에서 선언할 때
    std::cout << name << “/” << age << endl;
}
```

class

- static과 const 예제

```
class Circle {
private:
    static double radius;
public:
    double getRadius() const { return radius; } // Getter: 값을 읽기만 하므로 const 가능
    void setRadius(double r) { radius = r; } // Setter: 값을 수정해야 하므로 const 제거
    // 일반 멤버 함수: radius에 접근해야 하므로 static 제거
    static double getAreaOfCircle() { return radius * radius * 3.14; }
};
int main() {
    Circle c1;
    c1.setRadius(10.0);
    cout << "반지름: " << c1.getRadius() << endl;
    // 함수 호출 시 반드시 ()를 붙여야 함
    cout << "넓이: " << c1.getAreaOfCircle() << endl;
}
```

// static은 같은 static 끼리만
// 인스턴스는 상관없음.
// static붙으면 const 제거

class

- 멤버 함수 선언 – 인라인 함수
- 함수 본문이 짧을 때 성능 향상을 위해 컴파일러가 호출을 실제 코드로 대체, 최근에는 컴파일러가 알아서 처리함.

```
class Circle {
private:
   double radius;
public:
   double getRadius() const { return radius; }
}

// 명시적 인라인 함수 선언
inline double Circle::getRadius() const {
   return radius;
}
```

class

- 정적 전역 변수/함수 (Internal Linkage)

```
static int counter = 100;
static void internalLog() {
    cout << "내부 로그" << endl;
}
```

```
int main() {
    cout << "counter : " << counter << endl;
    internalLog();
}
```

- 다른 파일에서 접근 시도

file1.cpp

```
#include <iostream>
using namespace std;

static int value = 10;
```

file2.cpp

```
#include <iostream>
using namespace std;

extern int value;   // 접근 시도 error
                       static → internal linkage
                       → 다른 번역 단위(.cpp)에서 보이지 않음

int main() {
    cout << value << endl;
}
```

class

- 클래스 정의 후 사용하는 방법(인스턴스 化)

```
class Circle {
private:
   double radius;
public:
   double getRadius() const { return radius; } // Getter: 값을 읽기만 하므로 const 가능
   double setRadius(double r) { radius = r; } // Setter: 값을 수정해야 하므로 const 제거
}
int main(){
   Circle c1;  // 인스턴스 만듦
   // cout << c1.radius << endl; // 데이터 멤버가 private라서 접근 불가
   c1.setRadius(10.0);
   cout << c1.getRadius() << endl;
}
```

class

- 구조체 - C 언어에서 있었던 구조체를 C++에서 사용
- 구조체의 모든 멤버는 기본적으로 public, 클래스는 private
- 멤버 함수를 사용하지 않는 경우, 쉽게 만들고 요소에 직접 접근 가능한 구조체를 사용하기도 함.

```
struct Name {
   string first;
   char middle;
   string last;
};
```

=

```
class Name {
   public:
     string first;
     char middle;
     string last;
};
```

class

- 생성자 : 특별한 멤버 함수.
  - ✓ 클래스를 초기화하는데 사용 즉 인스턴스를 만들 때 사용
  - ✓ 리턴값이 없으며, 이름이 클래스 이름과 동일.
  - ✓ 매개변수가 있는 생성자, 기본 생성자
  - ✓ 위의 2 생성자가 없으면 합성 기본 생성자를 자동 생성.
  - ✓ 합성 기본 생성자는 데이터 멤버를 지정 안하면 쓰레기 값으로 초기화

```
class Circle {
private:
   double radius;
public:
   Circle(double radius);        // 매개변수가 있는 생성자
   Circle()                      // 기본 생성자
   Circle(const Circle& circle); // 복사 생성자
}
```

class

- 생성자
  - ✓ 복사 생성자는 기존 인스턴스와 똑같은 값을 가진 새로운 객체를 만듦.
  - ✓ 복사 생성자를 만들지 않으면 시스템이 합성 복사 생성자를 만듦.

```
class A {
public:
   int x;
   A(int value) {
      x = value;
      cout << “일반 생성자" << endl;
   }
   A(const A& other) {
      x = other.x;
      cout << “복사 생성자" << endl;
   }
};
```

```
int main() {
   A a1(10);     // 원본 객체 생성
   A a2 = a1;    // 복사 생성자 호출
   cout << "a1 value: " << a1.x << endl;
   cout << "a2 value: " << a2.x << endl;
   cout << "a1 address: " << &a1 << endl;
   cout << "a2 address: " << &a2 << endl;
}
```

class

- 생성자 초기화 리스트
  - ✓{ radius = 0.0; } (대입) //변수가 쓰레기값으로 생성된 후, 값을 **대입**함
  - ✓: radius(0.0) (초기화 리스트) //변수가 생성됨과 동시에 값을 초기화함

```
Circle::Circle(double rds)
: radius(rds) //초기화 리스트
{ }
Circle::Circle()
: radius(1.0) //생략하면 쓰레기 값 생성
{ }
Circle::Circle(const Circle& cr)
: radius(cr.radius) //초기화 리스트
{ }
```

```
class Circle {
private:
    const double PI;//상수 반드시 초기화 리스트 필요
    double radius;

public:
    // 초기화 리스트에서 PI를 3.14로 초기화
    Circle() : PI(3.14), radius(0.0) {
        // PI = 3.14; 라고 하면 에러(상수 대입 불가)
    }
};
```

class

- 생성자
  - ✓ Singleton Pattern : 프로그램 전체에서 **단 하나의 인스턴스만 존재**함

```
class Singleton {
private:
    Singleton() {} // 생성자를 private으로 숨김
public:
    // 복사 생성자와 복사 대입 연산자를 삭제(delete)하여 복사를 원천 봉쇄
    Singleton(const Singleton& extra) = delete;
    Singleton& operator=(const Singleton& extra) = delete;
    static Singleton& getInstance() {
        static Singleton instance;
        return instance;
    }
};
```

```
int main() {
    Singleton& s1 = Singleton::getInstance();
    Singleton& s2 = Singleton::getInstance();
    cout << "s1 주소: " << &s1 << endl;
    cout << "s2 주소: " << &s2 << endl;
    return 0;
}
```

class

- 소멸자
  - ✓ 리턴값이 없으며, 클래스 이름 앞에 ~가 붙음.
  - ✓ 만들어진 인스턴스의 소멸할 때 그 자원을 정리.
  - ✓ 소멸자는 public만 가능하다.
  - ✓ 만약 만들지 않으면 시스템이 합성 소멸자를 만듦.

```
class Circle {
private:
   double radius;
public:
   ...
   ~Circle()
}
Circle::~Circle() {
   문장
}
```

class

- 접근자 멤버 함수(Getter)
  - ✓ 읽기 전용 함수이며, 변경하지 않음을 확실히 할 때 const 붙임.

```
double getRadius() const;
double getPerimeter () const;
double getArea () const;
```

- 설정자 멤버 함수(Setter)
  - ✓ 설정자에는 const 한정자를 붙이지 않음.

```
void setRadius(double rds);
```

class

- this
  - ✓ this->radius와 (*this).radius는 같다.
  - ✓ 암묵적 사용

```
double getRadius() const {
    return radius;
}
// 사용자가 작성한 코드
```

```
double getRadius(Circle * this) const {
    return (this->radius);
}
// 컴파일러가 매개변수를 추가한 코드
```

  - ✓ 명시적 사용

```
void Circle::setRadius(double rds) {
    radius = rds;
}
// this를 사용하지 않은 코드
```

```
void Circle::setRadius(double radius) {
    this->radius = radius;
}
// this를 사용한 코드
```

class

- 캡슐화: 정보 은닉, 데이터 보호, 유지보수 향상, 결합도 감소
- 방법: 클래스 정의, 접근 제한자, Getter/Setter 사용
- 객체지향개념의 목표, 재 사용성의 극대화를 이룰 수 있음.

```
class Person {
private:
    std::string name; // 외부에서 직접 접근 불가
    int age;
public:

    Person(std::string n, int a) : name(n), age(a) {} //매개 변수있는 생성자
    int getAge() { return age; } // 안전한 데이터 접근을 위한 Getter
    void setAge(int a) {if (a > 0) age = a;} // 유효성 검사를 포함한 Setter

};
```

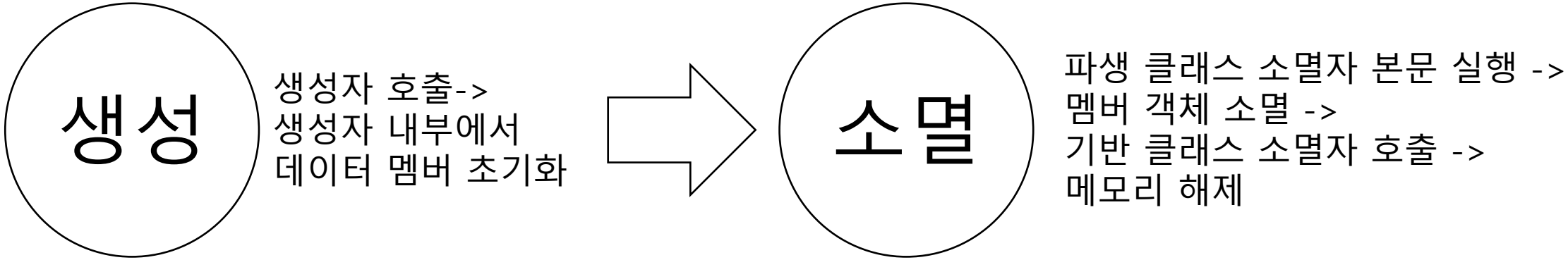
class

- 파일 분할의 3가지
  - ✓인터페이스 파일: 클래스 정의 포함된 파일,헤더파일 확장자 *.h
  - ✓구현 파일: 인터페이스 파일에는 모든 선언을 입력, 확장자*.cpp
  - ✓애플리케이션 파일: main함수가 포함된 파일, 확장자*.cpp
- 분할 컴파일: 모든 파일을 컴파일해서 실행 파일을 만들 때
- 전처리 지시문

```
# ifndef CIRCLE_H //플래그 정의 안되었다면, 헤더 파일 재열람.

#endif
```

class

- Life Cycle
  - ✓자동(Automatic) 객체: 함수내 지역 변수, 블록 {}을 벗어나면 자동 소멸.
  - ✓정적(Static) 객체: 함수 내 static 변수는 함수가 끝나도 값이 유지.
  - ✓동적(Dynamic) 객체: new 연산자로 Heap 메모리에 생성, delete를 통해 명시적으로 소멸.
  - ✓임시(Temporary) 객체: 표현식에서 일시적으로 생성되며, 해당 표현식이 끝나면 즉시 소멸합니다.



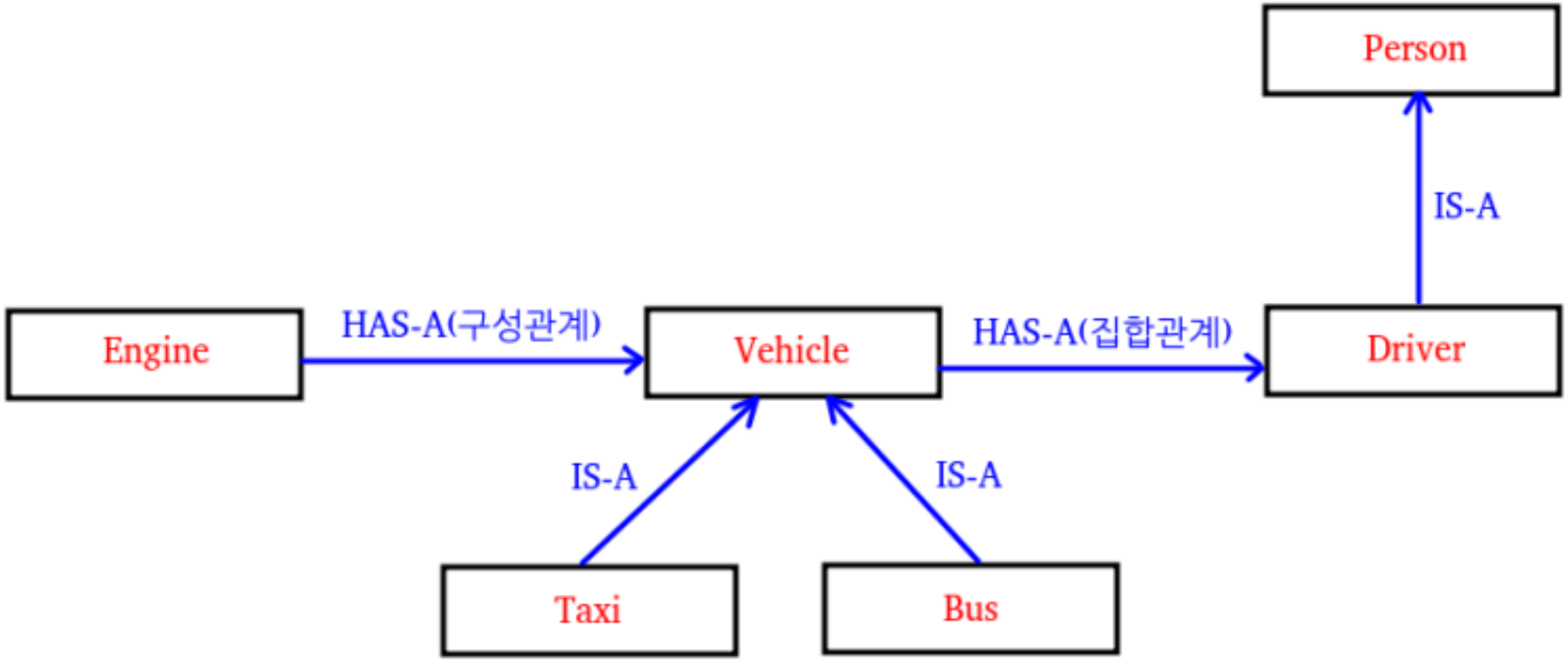
class

- overriding
  - ✓ toString( )

```
class Circle {
private:
    double radius = 5.0;
public:
    // << 연산자를 오버로딩하여 cout이 Circle 객체를 다룰 수 있게 함
    friend ostream& operator<<(ostream& os, const Circle& c) {
        os << "Circle(반지름: " << c.radius << ")";
        return os;
    }
};
int main() {
    Circle c1;
    // 이제 컴퓨터가 c1을 어떻게 출력할지 알게 됨
    cout << c1 << endl;
    return 0;
}
```

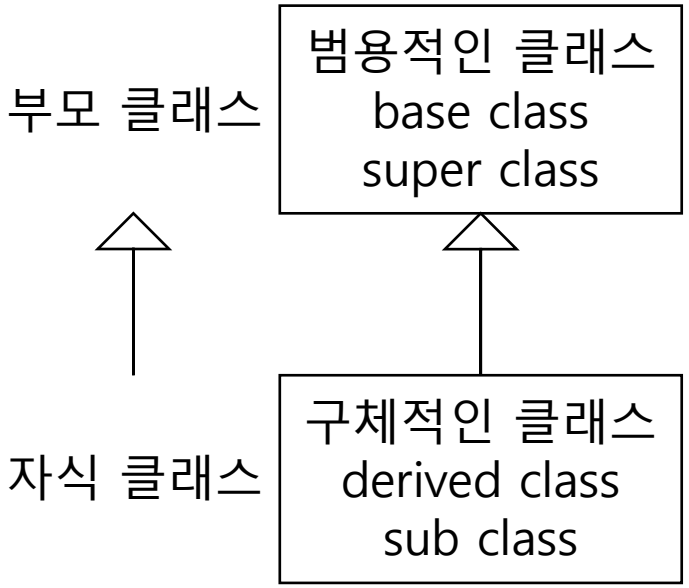
class

- 클래스의 관계: is-a 관계, has-a 관계



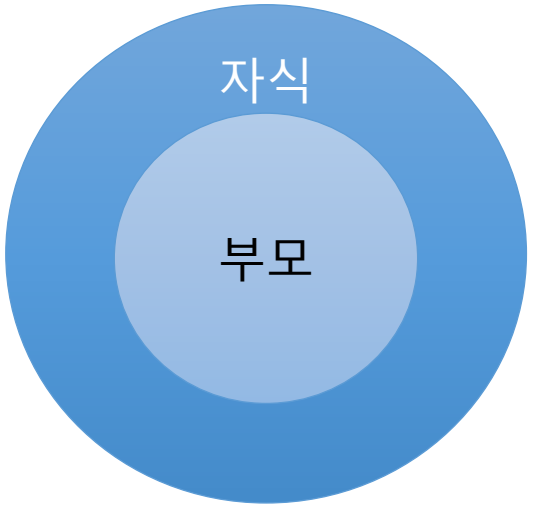
class 상속

- 부모클래스는 생성자,소멸자,할당연산자를 제외한 나머지 상속
- 상속의 종류를 지정하지 않으면 private, 일반적 public 사용
- 부모의 private 속성은 상속은 되나 자식에서는 접근 불가




```
class Parent{
    //...
}
```

```
class Child : public Parent{
  // public, protected, private
}
```



자식 ⊇ 부모

class 상속

- 부모 속성에 값을 적용하는 3가지 방법(세터,초기화리스트,위임)
  - ✓ 생성자 초기화 리스트

```
class Person{
private:
  long id;
public:
  void setId(long id);
  long getId() const;
  Person(id){this->id=id;}
};
```

```
class Student : public Person{
private:
  double gpa;
public:
  void setGPA(double gpa);
  double getGPA() const;
  Student(long id):Person(id){}
};
```

```
int main() {
    Person person;
    person.setId(11111L);
    Student student;
    student.setId(22222L);
}
```

  - ✓ 위임

```
void Student::set(long id, double gp) {
    Person::setId(id);
    gpa = gp;
}
```

```
void Student::print() {
    Person::print();
    cout << gpa;
}
```

class 상속

- 멤버 함수의 Overloading

```
void set(long identity);
void set(double gpa);
```

- 멤버 함수의 Overriding

```
long get()  // Person
double get() // Student
```

- 기본 생성자, 매개변수 있는 생성자, 복사생성자, 소멸자, 할당 연산자라는 5개의 멤버 함수는 파생클래스로 상속 되지 않는다.
- 소멸자에서 private 변수는 소멸시킬 수 없지만 자식클래스에서 부모 클래스의 생성자를 호출하고 마찬가지로 자식클래스에서 부모클래스의 소멸자를 호출하여 소멸시킬 수 있다.

class 상속

- 위임

```
// 조상 클래스
Person::Person()
: identity(0)
{}

Person::Person(long id)
: identity(id)
{}

Person::Person(const Person& obj)
: identity(obj.identity)
{}

Person::~Person()
{}
```

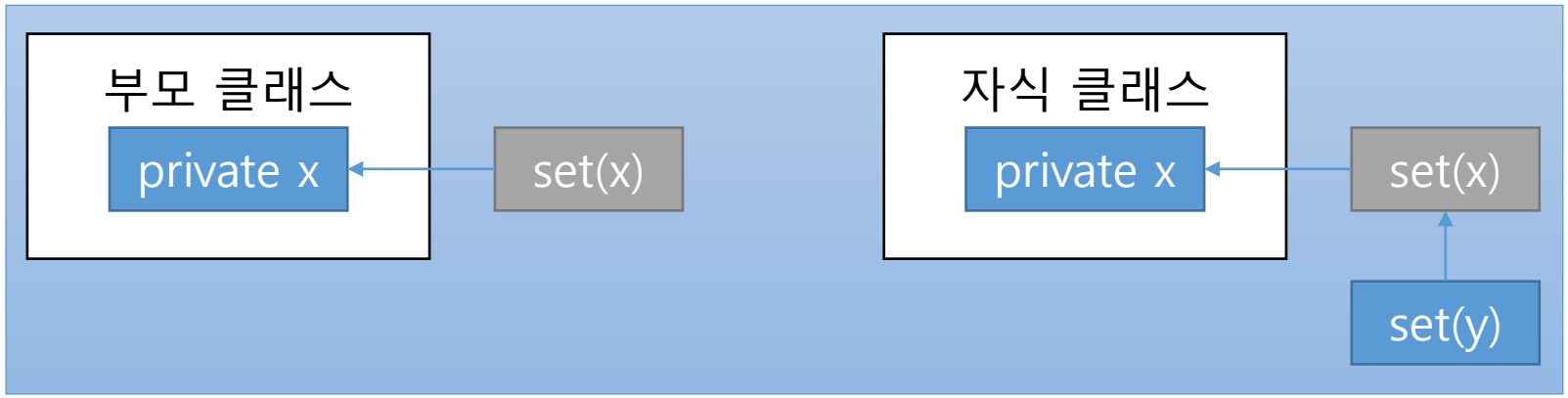
```
// 자식 클래스
Student::Student()
: Person(0), gpa(0.0)
{}

Student::Student(long id, double gp)
: Person(id), gpa(gp)
{}

Student::Student(const Student& st)
: Person(st), gpa(st.gpa)
{}

Student::~Student()
{}
```

class 상속

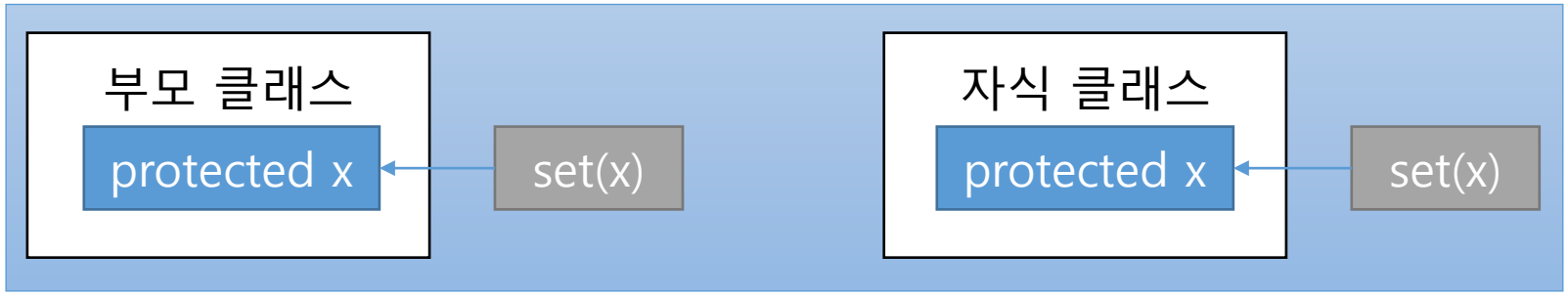
- 상속의 접근과 막기 - protected

```
void 자식::set(int y) {
  부모::set(y);
} //부모에 접근
```

상속 때의 private 멤버



상속 때의 protected 멤버
public 처럼 사용가능



class 상속

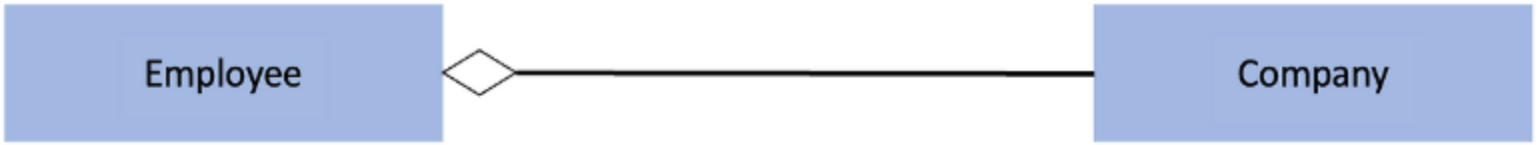
- 상속의 접근과 막기 – final

```
calss First Final
{ ...
}
```

```
Class First
{ ...
}

class Second final : public First
{ ...
}
```

class 연관

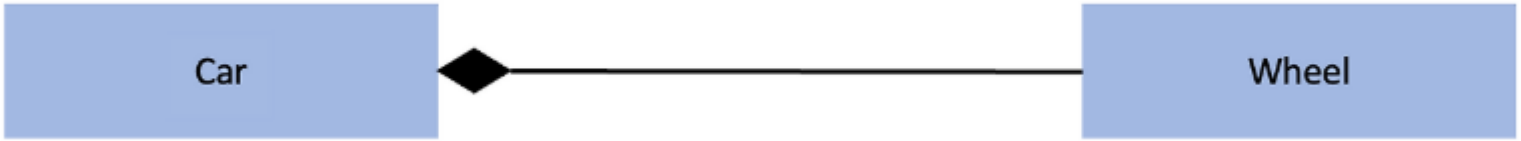
- 소유
  has-a




```
class Employee {
  // ...
};

class Company {
private:
  // 직원 객체에 대한 포인터를 저장하는 vector
  std::vector<Employee*> employees;
public:
  // 직원 객체를 추가하는 메서드
  void addEmployee(Employee* employee) {
    employees.push_back(employee);
  }
};
```

class 연관

- 구성
  has-a




```
class Wheel {  // ... };
class Car {
  private:
    Wheel* wheels[4]; // 바퀴 객체에 대한 포인터를 저장하는 배열
  public:
    Car() {  // 생성자에서 바퀴 객체를 생성
       for (int i = 0; i < 4; i++) {
        wheels[i] = new Wheel();
      }
    }
    ~Car() {  // 소멸자에서 바퀴 객체를 삭제
      for (int i = 0; i < 4; i++) {
        delete wheels[i];
      }
    }
};
```

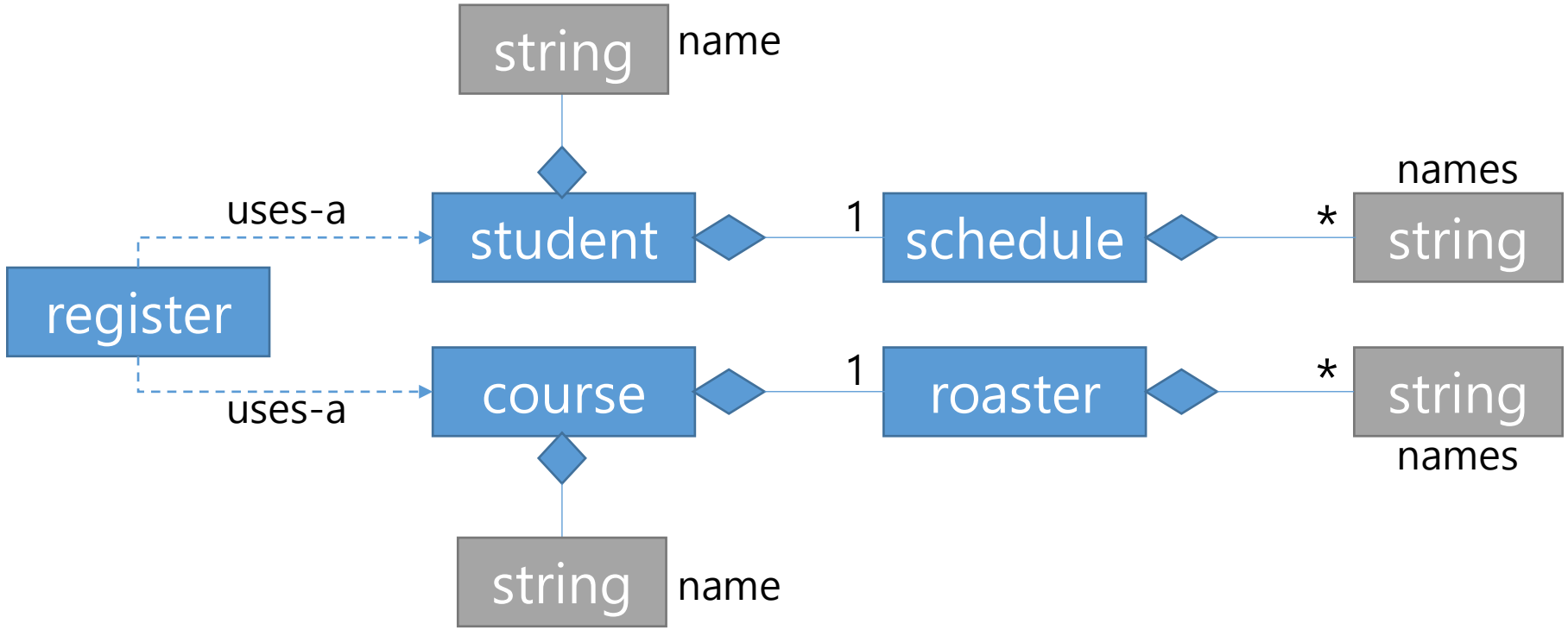
class 의존

- 사용
  uses-a

- 의존 관계는 상속과 연관 보다는 약한 관계

class 응용

- 대학교 수강과목 프로그램 – 연관, 의존 관계를 모두 살펴 보자



class 인스턴스 비교

```
class Base { };

int main() {
    Base* ptr = new Base();
    Base b;
}
```

구분	Base* ptr = new Base();	Base b;
변수 타입	포인터 변수 (Base*)	객체 변수 (Base)
생성 방식	new 연산자로 객체 생성	직접 객체 생성
메모리 위치	Heap(동적 메모리)	Stack(자동 메모리)
접근 방식	ptr->멤버	b.멤버
메모리 해제	delete ptr; 필요	자동 해제
수명	delete 할 때까지	블록 {} 종료 시

```
Base* ptr = new Base();
```

주로 다형성(polymorphism) 에 사용

```
class Child : public Base {};
Base* ptr = new Child();   // 가능
```

class 다형성

- 다형성(Polymorphism) : 같은 이름을 갖는 여러 형태의 함수를 클래스별로 만들 수 있게 해주는 기능, 동적 바인딩과 관련.
- virtual
  - ✓상속 관계의 두 인스턴스가 공통으로 가지는 함수를 각각의 인스턴스의 내용에 맞게 사용가능하게 함.
  - ✓생성자는 불가, 소멸자는 가능
  - ✓virtual + 포인터 다형성 (정상적인 다형성)
  - ✓virtual 때문에 실행 시 실제 객체 타입 확인

상황	결과
Base b = Child()	Base 함수 (슬라이싱)
Base* ptr = &Child + virtual 없음	Base 함수
Base* ptr = &Child + virtual 있음	Child 함수

class 다형성

// virtual + 포인터 다형성 (정상적인 다형성)
// virtual 때문에 실행 시 실제 객체 타입 확인

```
class Base {
public:
  virtual void show() {
    cout << "Base show" << endl;
  }
};

class Child : public Base {
public:
  void show() {
     cout << "Child show" << endl;
  }
};

int main() {
  Child c;
  Base b = c;    // 객체 슬라이싱
  b.show();       // Base show
}
```

```
class Base {
public:
    virtual void show() {
        cout << "Base show" << endl;
    }
};
class Child : public Base {
public:
    void show() {
        cout << "Child show" << endl;
    }
};

int main() {
    Base* ptr;
    Child c;
    ptr = &c;
    ptr->show(); // Child show
}
```

class 다형성 //상속관계를 이용

```
class Product {
private:
  int price;  string name;
public:
  Product(int p, string n) : price(p),
name(n) {}
  int getPrice() const { return price; }
};

class Tv : public Product  {
public:
  Tv();
};
Tv::Tv() : Product(300, "TV") {}

class Computer : public Product {
public:
  Computer() :Product(300, "Computer") {}
};
```

```
class Customer {
private:
   int money = 1000;
public:
   void buy(Product& product) {
      money -= product.getPrice();
   }
   void showMoney() const {
       cout <<  " 잔돈: " << money << endl;
   }
};
int main() {
    Customer customer;
    Tv tv;  Computer com; Audio audio;
    customer.buy(tv);
    customer.buy(audio);
    customer.buy(com);
    customer.showMoney();
}
```

class

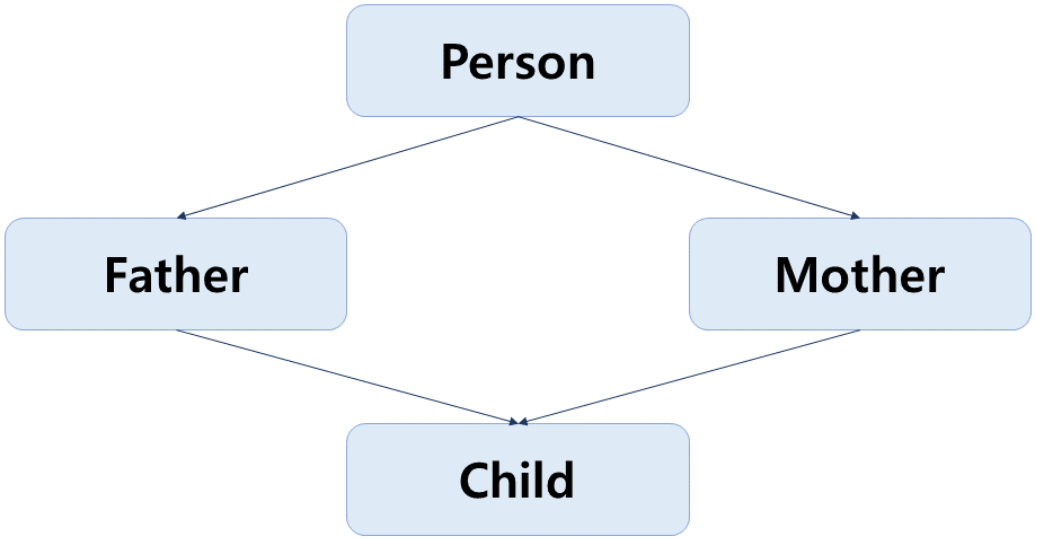
- 추상클래스
  - ✓하나 이상의 순수 가상 함수를 갖고 있어야 함.
  - ✓스스로 인스턴스 불가, 상속 받아서 재정의를 해야 함.

```
virtual double getArea() = 0;
virtual double getPerimeter() = 0;
```

- 인터페이스
  - ✓모든 멤버 함수를 순수 가상 함수로 가짐.
  - ✓스스로 인스턴스 불가, 상속 받아서 재정의를 해야 함.
  - ✓작업의 표준화를 위해 필요

class

- 다중 상속
- 둘 이상의 클래스를 조상클래스로 자식 클래스를 만드는 일



class

- 믹스인 클래스(mixin class)
- 순수 가상 함수를 가져서 인스턴스화는 안되지만, 다른 클래스에 데이터 멤버를 추가해주는 클래스를 의미함.
- 딸기 와플, 포도 와플에 생크림 토핑을 준다고 할 때 토핑은 믹스인 클래스에 해당함.

예외처리

- 프로그램 실행 중 발생할 수 있는 오류 상황을 안전하게 처리하는 방법
- 장점: 파일이 없을 때, 메모리 부족, 잘못된 값 입력 같은 상황에서 사용
  - ✓코드 안정성 : 프로그램 종료 방지
  - ✓에러 분리 : 정상 코드와 에러 코드 분리
  - ✓상위 함수 처리 : 함수에서 던지고 main에서 처리
- 대표적인 Exception 종류
  1. exception
  2. runtime_error
  3. logic_error
  4. out_of_range
  5. invalid_argument
  6. bad_alloc

```
int main() {
  try {
      throw 10;   // 예외 발생
  }
  catch (int e) {
      cout << "예외 발생: " << e << endl;
  }
  cout << "프로그램 계속 실행" << endl;
}
```

Array

- 같은 形의 여러 값을 연속된 메모리 공간에 저장하는 자료구조
- 단점 : 크기가 고정, 동일한 자료형
- 선언: int arr[5]; (쓰레기 값으로 초기화)
- 초기화: int arr[5] = {1, 2, 3, 4, 5};
- 부분 초기화: int arr[10] = {1, 2}; (나머지는 0으로 초기화)
- 전체 0 초기화: int arr[10] = {0};
- 크기 자동 지정: int arr[] = {1, 2, 3}; (크기가 3으로 자동 설정)

제네릭(Generic)

- 데이터 type에 의존하지 않고, 템플릿(template)을 사용하여 함수나 클래스를 일반화하여 코드를 재사용하는 프로그래밍 방식.
- 제네릭 함수 (Function Template), 제네릭 클래스 (Class Template), STL(Standard Template Library) 활용

```
#include <iostream>
template <typename T>
T add(T a, T b) {
    return a + b;
}

int main() {
    std::cout << add<int>(3, 4) << std::endl;       // 7 (int로 구체화)
    std::cout << add<double>(3.5, 4.5) << std::endl; // 8.0 (double로 구체화)
    return 0;
}
```

STL (Standard Template Library)

- 배열의 단점(정해진 사이즈, 동일 타입 등)을 극복하기 위함.
- 검증된 데이터 구조와 알고리즘을 제공하여 개발 속도를 높이고, 코드 재사용성을 극대화하며, 높은 성능과 안정성을 보장

종류	C++ STL	Java Collection
동적 배열	vector	ArrayList
연결 리스트	list	LinkedList
스택	stack	Stack
큐	queue	Queue
덱	deque	ArrayDeque
집합	set	Set
맵	map	Map / HashMap

입출력 IO

- 프로그램이 **외부(사용자, 파일 등)와 데이터를 주고받는 기능**
  - ➢ifstream (Input File Stream): 파일로부터 데이터를 읽을 때 사용합니다.
  - ➢ofstream (Output File Stream): 파일에 데이터를 쓸 때 사용합니다.
  - ➢fstream: 읽기와 쓰기를 동시에 할 때 사용합니다.

단계	함수 / 연산자	설명
열기	open("파일명")	파일을 스트림에 연결합니다.
체크	is_open()	파일이 성공적으로 열렸는지 확인 (불린 값 반환)
쓰기	<< (삽입 연산자)	cout처럼 데이터를 파일에 저장합니다.
읽기	>> 또는 getline()	단어 단위나 줄 단위로 데이터를 가져옵니다.
닫기	close()	사용이 끝난 파일을 닫고 자원을 반환합니다.

쓰레드 Thread

- 하나의 프로그램 안에서 **동시에 실행되는 작업 흐름**
  - ➢다운로드 하면서 화면 표시
  - ➢게임에서 AI 계산 + 렌더링
  - ➢서버에서 여러 요청 처리

```
#include <iostream>
#include <thread>
using namespace std;

void work() {
  cout << "Thread 실행중" << endl;
}

int main() {
  thread t(work); // 쓰레드 생성
  t.join(); // 쓰레드 종료까지 대기
  cout << "main 종료" << endl;
}
```